02

Part 2 · Partitions & Data Movement

How data is split, moved and skipped: the biggest levers of performance.

- Partitioning
- Narrow vs wide
- Shuffling
- repartition vs coalesce
- Shuffle partitions
- Partition pruning · DPP
- Column pruning
- Predicate pushdown
- Partition strategy

2.1 Data Partitioning

★★★

IN ONE LINE

Spark splits data into partitions so many tasks can process it in parallel.

A **partition** is a slice of a dataset that lives on one executor and is processed by **one task**. Partitioning is the root of Spark performance: it decides **how much parallelism** you get and **how evenly** work is shared.

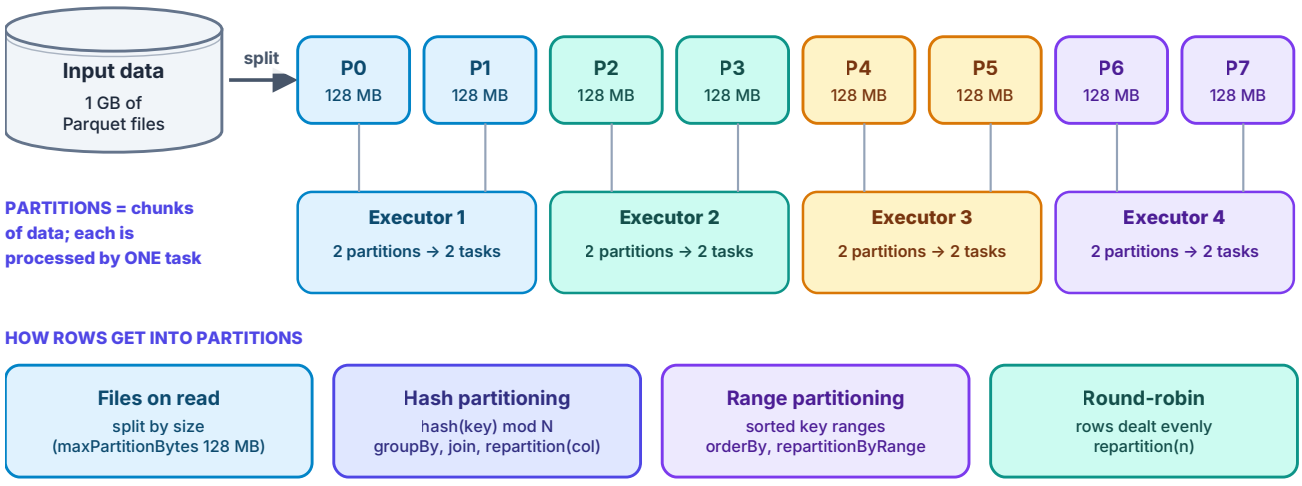


Figure 2.1 — Data split into partitions, spread over executors

Question	Answer
How many partitions on read?	Input size ÷ `spark.sql.files.maxPartitionBytes` (default **128 MB**), subject to file boundaries.
How many after a shuffle?	`spark.sql.shuffle.partitions` (default **200**), or fewer if AQE coalesces them (6.2).
What is a good partition size?	Roughly **100–200 MB** in memory. Tiny = overhead; huge = spill and OOM.
How many partitions per core?	**2–4**, so cores never idle and stragglers are absorbed.
How do I check?	`df.rdd.getNumPartitions()` or the task count in the Spark UI.

INSPECT PARTITIONING

```
df = spark.read.parquet('/lake/bronze/events')
print(df.rdd.getNumPartitions())      # e.g. 64

from pyspark.sql.functions import spark_partition_id
df.groupBy(spark_partition_id().alias('pid')).count().show()   # rows per partition: spot uneven data
```

REMEMBER

Partitioning in memory (how Spark splits data while computing) is different from **partitioning on disk** (`partitionBy` folders, 2.9). Interviewers often probe whether you know the difference.

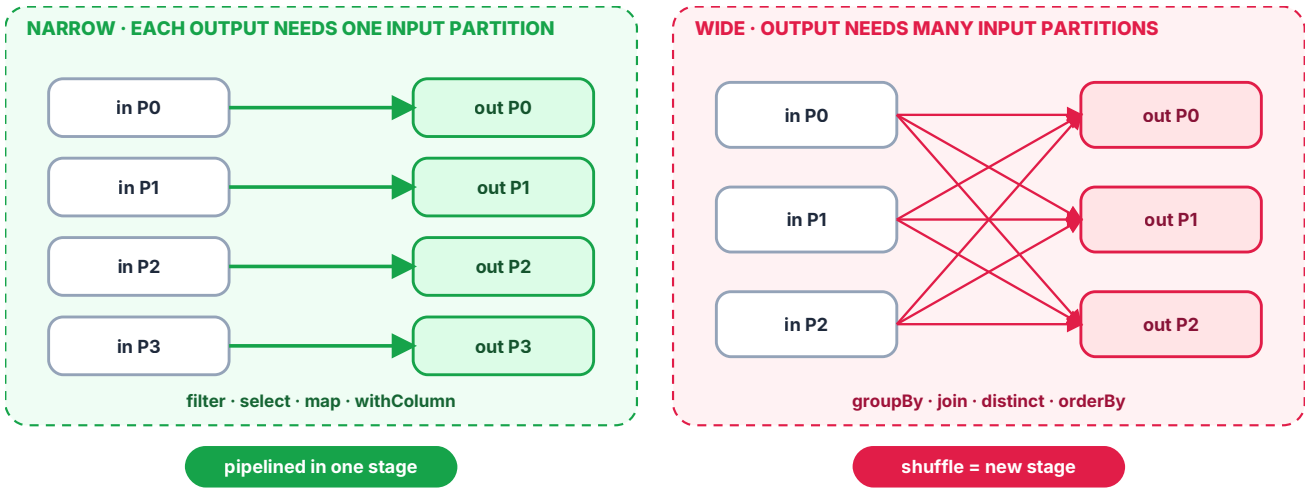
2.2 Narrow vs Wide Transformations

★★★

IN ONE LINE

Narrow transformations keep data in place; wide ones require a shuffle.

A **narrow transformation** computes each output partition from **one** input partition, so partitions stay where they are. A **wide transformation** needs data from **many** partitions, which forces a **shuffle** (all-to-all exchange).



Narrow dependencies are cheap and fault tolerant; wide dependencies are where time and money go.

Figure 2.2 — Narrow = 1-to-1, wide = all-to-all

	Narrow	Wide
Examples	`select`, `filter`, `withColumn`, `map`, `union`, `coalesce`	`groupBy`, `join` (non-broadcast), `distinct`, `orderBy`, `repartition`, window with `partitionBy`
Data movement	None	Across the network
Stages	Chained inside one stage	Each wide op ends a stage
Failure recovery	Recompute one parent partition	May need many parents (or shuffle files)
Cost	Low	**High**

INTERVIEW ANGLE

The optimisation mindset: **keep as much work narrow as possible, and make each wide step as small as possible** (filter and project first, broadcast small tables, avoid repeated groupBys).

2.3 Shuffling

★★★

IN ONE LINE

The all-to-all movement of data between executors, and the main cost in Spark jobs.

Shuffling is the redistribution of data across partitions so that rows that belong together (same join key or group key) end up on the same executor. It is the **single most expensive operation** in most Spark jobs.

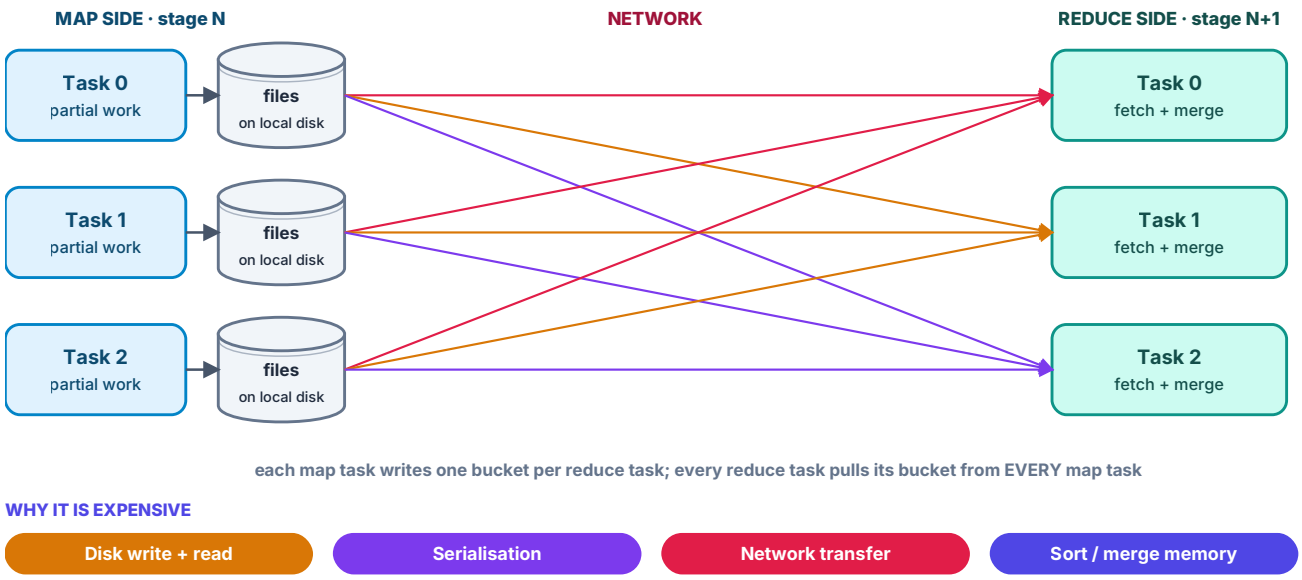


Figure 2.3 — Map tasks write shuffle files; reduce tasks fetch them across the network

What triggers a shuffle

- `groupBy`, `agg`, `distinct`
- `join` (unless broadcast)
- `orderBy`, `repartition`
- window functions with `partitionBy`

How to shuffle less

- **Filter and select first** to shrink data.
- **Broadcast** small tables (3.2).
- Aggregate in **one** pass, not many.
- Avoid unnecessary `repartition` / `orderBy`.
- Use partitioned / bucketed data to align keys.
- Tune shuffle partitions (2.5).

REMEMBER

In the Spark UI a stage with large **Shuffle Write** feeds a stage with large **Shuffle Read**. If shuffle is large, the next questions are: *can I reduce the data first? can I avoid it? is it skewed?*

INTERVIEW ANGLE

A classic interview line: "Shuffle is expensive because it involves **disk I/O, serialisation and network transfer**, and creates a stage boundary."

2.4 repartition() vs coalesce()

★★★

IN ONE LINE

repartition shuffles to rebalance (up or down); coalesce cheaply merges partitions (down only).

Both change the number of partitions, but they work very differently.

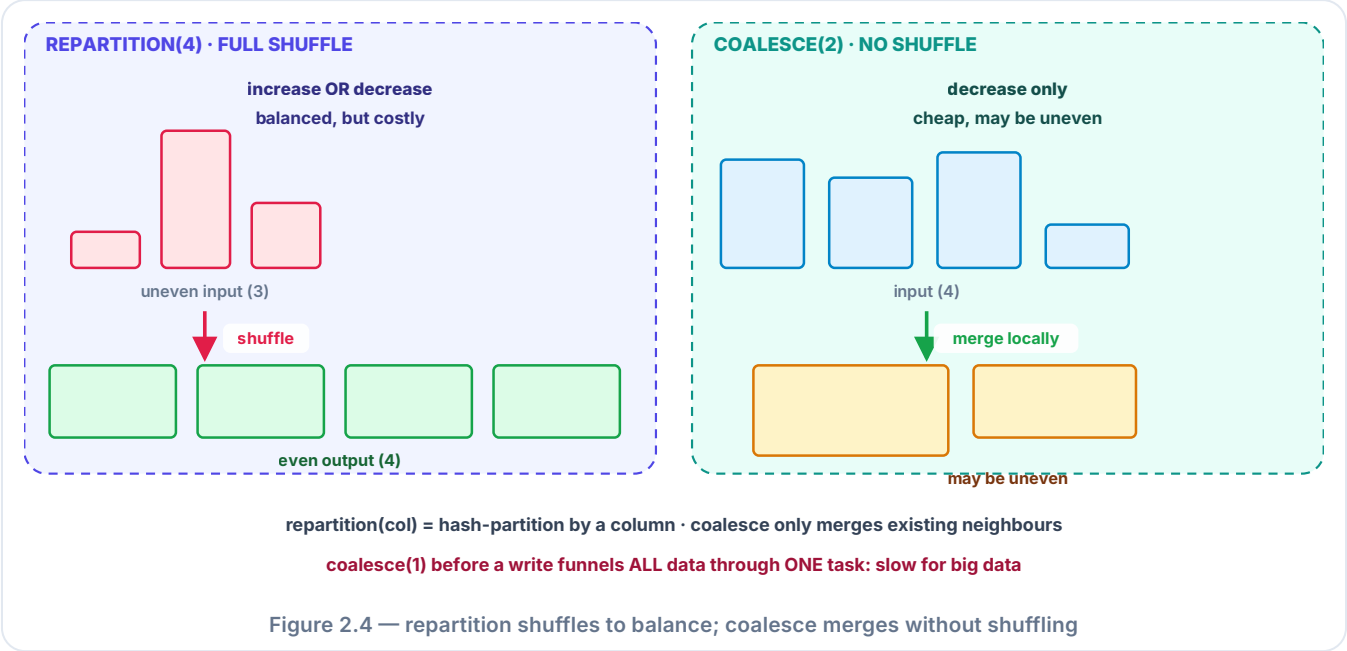


Figure 2.4 — repartition shuffles to balance; coalesce merges without shuffling

	`repartition(n)`	`coalesce(n)`
Shuffle	**Yes**, full	**No** (merges neighbours)
Can increase partitions	Yes	No (decrease only)
Output balance	Even	Possibly uneven
Cost	High	Low
Use when	Need more parallelism, fix skew, partition by column before a join / write	Reduce number of output files / tasks after heavy filtering

TYPICAL USAGE

```
# fewer, bigger output files after a selective filter
(df.filter('country = "IN"')
   .coalesce(8)
   .write.mode('overwrite').parquet('/lake/silver/in_orders'))

# more parallelism, balanced, keyed for a downstream join
df = df.repartition(400, 'customer_id')

# one file per date value
df.repartition('order_date').write.partitionBy('order_date').parquet(path)
```

COMMON TRAP

`coalesce(n)` can **reduce the parallelism of the whole stage before it** (Spark may push the smaller partition count upstream). If you filtered a lot and then need heavy computation, `repartition` is safer. **Never `coalesce(1)` on big data** except for tiny outputs.

2.5 Shuffle Partition Tuning

★★★

IN ONE LINE

Choosing spark.sql.shuffle.partitions so shuffle tasks are neither huge nor tiny.

`spark.sql.shuffle.partitions` sets how many partitions **every shuffle** (join, groupBy, ...) produces. Its default is **200** regardless of data size, which is wrong for both small and huge jobs.

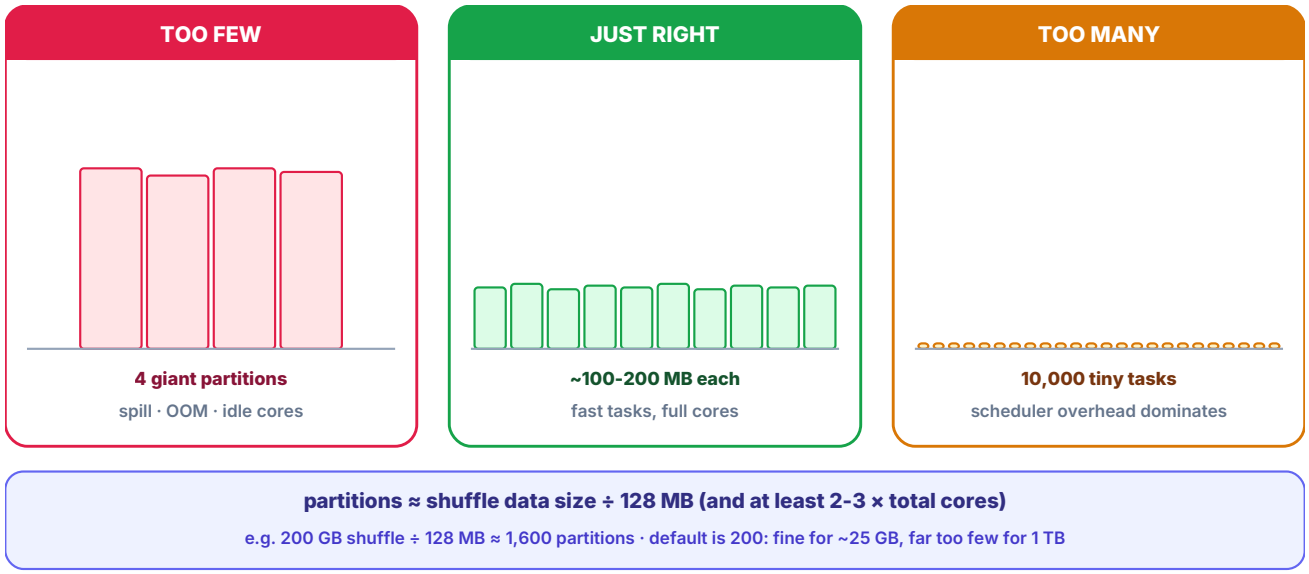


Figure 2.5 — Too few, just right, too many shuffle partitions

Symptom in Spark UI	Diagnosis	Action
Few huge tasks, spill to disk, OOM, many idle cores	Too few partitions	Increase shuffle partitions
Thousands of ~ms tasks, long scheduling gaps, many tiny output files	Too many partitions	Decrease (or let AQE coalesce them)
One task much slower than the rest	Not a count problem: **skew**	3.5 – 3.7

TUNING SHUFFLE PARTITIONS

```
# set per job or per session
spark.conf.set('spark.sql.shuffle.partitions', 800)

# modern default: let AQE pick the right number at runtime (Spark 3.2+/Databricks)
spark.conf.set('spark.sql.adaptive.enabled', True)
spark.conf.set('spark.sql.adaptive.coalescePartitions.enabled', True)
```

INTERVIEW ANGLE
With **AQE** on (default since Spark 3.2), set shuffle partitions **generously high** and let Spark **coalesce** small ones at runtime (6.2). In **Structured Streaming** the value is frozen in the checkpoint, so choose carefully before the first run (8.3).

2.6 Partition Pruning

★★★

IN ONE LINE
Skipping whole partition folders when a filter matches the partition column.

If a table is stored in folders by a column (`year=2024/month=03/`), a filter on that column lets Spark **skip entire folders** without opening them. This is the cheapest optimisation there is: data that is never read costs nothing.



Figure 2.6 — Static partition pruning and dynamic partition pruning

	Static pruning	Dynamic partition pruning (DPP)
Filter on	The partition column directly	A **dimension table** filter joined to the fact table's partition column
Decided	At planning time	At runtime (after the small side is evaluated)
Needs	Filter written on the partition column, no function around it	Star-schema join, fact table partitioned on the join key; enabled by default

COMMON TRAP
Wrapping the column in a function **defeats pruning**: `WHERE year(order_date) = 2024` may scan everything, while `WHERE order_date >= "2024-01-01" AND order_date < "2025-01-01"` prunes. Also compare with the **same data type**: a string vs int mismatch can disable pruning.

INTERVIEW ANGLE
Verify with `explain()` : look for **PartitionFilters** in the FileScan node and a small number of files read in the Spark UI. Design the layout around the filters your queries use most (2.9).

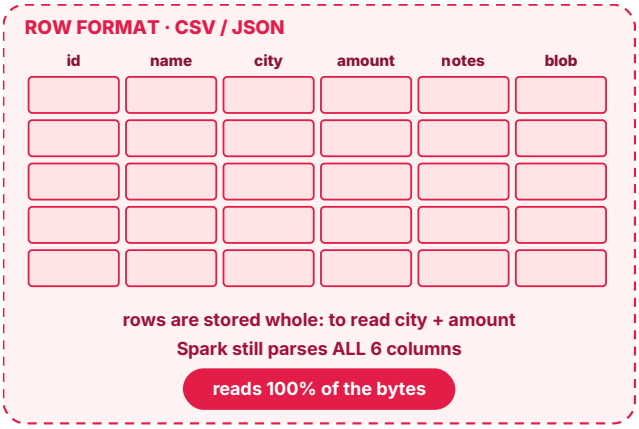
2.7 Column Pruning

★★★

IN ONE LINE

Reading only the columns a query needs, which columnar formats make nearly free.

Column pruning means reading only the columns a query actually uses. With **columnar formats** like Parquet and ORC, each column is stored separately, so unused columns are never read from disk or sent over the network. Catalyst does this automatically — *if* the format and your query allow it.




```
spark.read.parquet(p).select('city', 'amount') # prunes
spark.read.parquet(p) # SELECT *: reads every column
```

Figure 2.7 — Row formats read everything; columnar formats read only what is needed

How to benefit

- Store analytical data as **Parquet / Delta**.
- **Select only needed columns** early; avoid `SELECT *`.
- Avoid `df.rdd` or UDFs over whole rows.
- Wide tables with 200+ columns benefit most.

How to verify

In `explain()`, the FileScan node shows **ReadSchema: struct<city, amount>** listing only the columns read. In the Spark UI SQL tab compare *size of files read*.

INTERVIEW ANGLE

Pair it with the previous two: **partition pruning** skips folders, **column pruning** skips columns, **predicate pushdown** skips row groups. Together they let Spark read a tiny part of a huge table.

2.8 Predicate Pushdown

★★★

IN ONE LINE

Pushing filters into the data source so it can skip data before Spark reads it.

Predicate pushdown sends your filter conditions down to the data source (Parquet, JDBC, Delta), which can skip data before Spark ever sees it. Parquet keeps **min/max statistics per row group**, so whole chunks can be skipped; a JDBC source can run the `WHERE` inside the database.

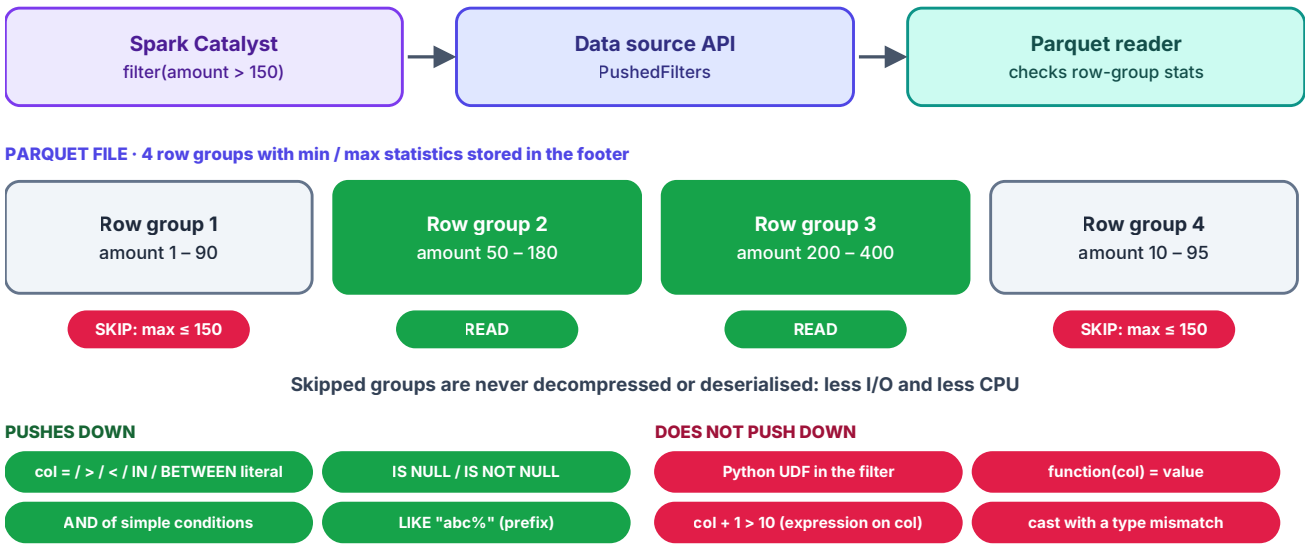


Figure 2.8 — Filters pushed to the reader skip row groups using min/max statistics

Pruning type	Skips	Works because of
Partition pruning	Folders	Directory layout (2.6)
Column pruning	Columns	Columnar storage (2.7)
Predicate pushdown	Row groups / rows	File statistics, or the source database

WRITE PUSHDOWN-FRIENDLY FILTERS

```
# good: simple comparison on a raw column pushes down
df.filter('amount > 150')

# bad: function on the column blocks pushdown and stats-based skipping
df.filter(F.round('amount') > 150)
df.filter(my_python_udf('amount'))
```

INTERVIEW ANGLE

Verify in `explain()` : **PushedFilters: [GreaterThan(amount,150)]** on the scan. Sorted or clustered data (Z-order, 5.7) makes min/max ranges narrow, so pushdown skips far more.

2.9 Choosing the Right Partition Strategy

★★★

IN ONE LINE

Pick low-cardinality, frequently filtered columns so partitions are large enough to be efficient.

On-disk partitioning (`write.partitionBy("col")`) creates one folder per value. Done well, it enables pruning; done badly, it creates the **small-files problem** (5.4).

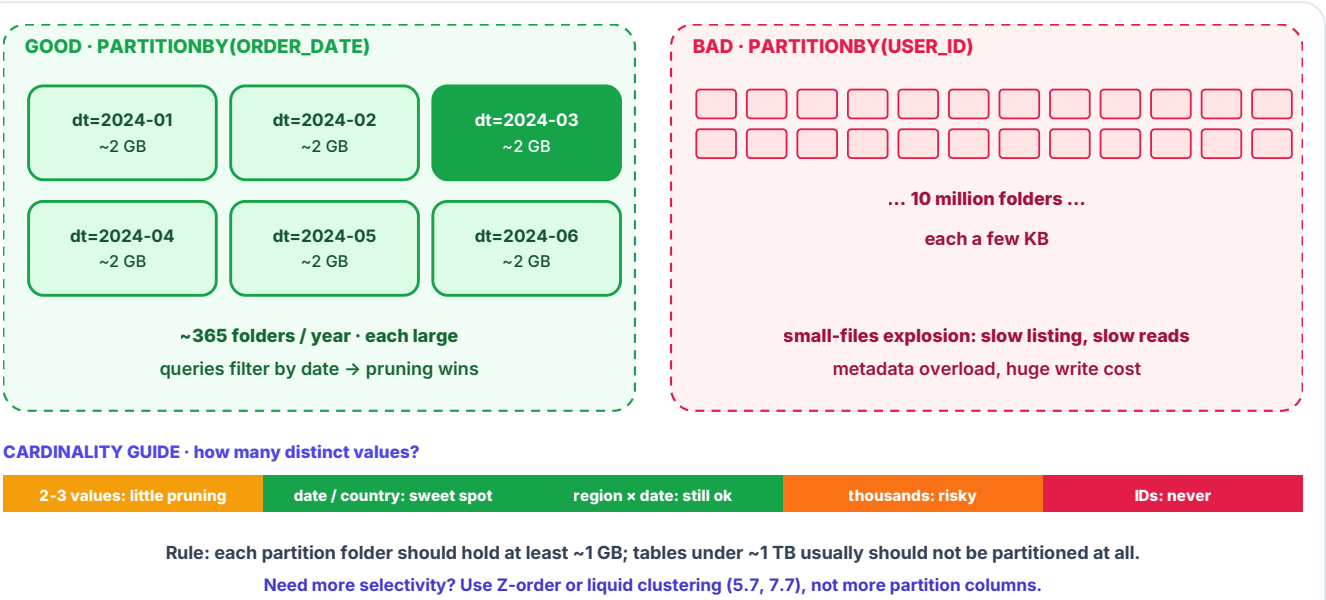


Figure 2.9 — Good vs bad partition columns, and a cardinality guide

Checklist	Why
Column is frequently filtered	Otherwise no query benefits from pruning.
Low to medium cardinality (date, country, month)	Keeps folder count manageable; avoid IDs, timestamps, emails.
Each partition ≥ ~1 GB; table > ~1 TB	Avoids tiny files and listing overhead.
Data is evenly spread across values	Skewed partition values give skewed tasks.
Avoid multi-level partitioning unless huge	year/month/day/hour multiplies folders fast.

Alternatives

- **Bucketing**: pre-hash data on a join key into N files to skip shuffles in repeated joins (Hive-style tables).
- **Z-ordering** and **liquid clustering** (Delta): data layout by several columns without partition folders.

In memory vs on disk

`repartition()` shapes *in-memory* partitions; `partitionBy()` shapes *folders on disk*. Combine them: `df.repartition("dt").write.partitionBy("dt")` writes **one file per folder**.

INTERVIEW ANGLE

Typical question: *"Would you partition by customer_id?"* No: high cardinality → millions of tiny files. Partition by **date**, and use clustering or bucketing for customer-level lookups.